WHOLESALE ERP / ORDER-TO-CASH PLATFORM

Living Design & Planning Document

Session 3 — February 26, 2026 | Status: Stakeholder Meeting Complete → Proposal Rebuild

---

🤖 CLAUDE'S BRIEFING — READ THIS FIRST

What we are doing: Designing a full wholesale food distribution ERP app from scratch, through a structured Q&A session. Claude asks questions, pulls everything out of the user, and documents it here in real time. This document is the single source of truth.

Session structure:

1. 🔵 **User Flow** ← WE ARE HERE — discovering all users, roles, needs, end-to-end flows
2. 🟡 App Design — screens, navigation, UX
3. 🟠 Infrastructure — tech stack, language, architecture
4. 🔴 Code Design — modules, classes, data models
5. 📋 Final Documentation — README, flow diagrams, API docs
6. 📅 Project Timeline — task list with durations

How to continue: Read this whole document to restore context, then pick up from the **Next Steps** section at the bottom. Ask the next question in the Q&A flow. Update this document after every exchange.

Current focus: Stakeholder meeting (Feb 25) answered most open questions. Living doc updated with all answers. Next: rebuild proposal in new format (concise, page-by-page script, required vs optional, pricing).

User: Wholesale food distribution company. Food industry (bottles, cans, packaged goods). Company also does manufacturing and packaging. Currently on-premises setup. Plans to eventually move app to cloud, keep DB on-prem.

KEY CHANGES FROM MEETING (Feb 25-26):

- 5-digit product codes (not 6), auto-generated with structured hierarchy
- No sales tax — wholesale only. CRV separate and not taxed
- Stock-based with manual/import updates, dynamic over-order rules
- Dynamic unlimited price levels (by product, category, customer, type, area, zip, custom groups)
- Salespeople collect payments in the field
- Warehouse basic = paper pick, manager manual entry. Digital pick = optional
- Driver can edit order before confirming (flags accounting + notifies salesperson)
- Digital signature option instead of printout for driver
- Basic offline included (device storage, sync on reconnect, conflict resolution)

> - Email notifications = base. SMS + WhatsApp = optional
> - AI features (voice, OCR, invoice review) all moved to optional or removed
> - Invoice review = rules-based price flagging (not AI)
> - Credit scoring system = new optional feature
> - Monthly statement lock (manual, like weekly invoice review)
> - New customers default to $0 credit / pay-on-order
> - Base price: $8,000 / 4 weeks
> - Reports: full report builder with sort/filter/join, standard reports, saved/scheduled reports, export + graphs = BASE
> - AI Reports (plain English → read-only queries, metadata-only AI access, Python sandbox) = OPTIONAL

📋 Session Roadmap

#	Phase	Status
1	User Flow	🔵 IN PROGRESS
2	App Design	⬜ Pending
3	Infrastructure	⬜ Pending
4	Code Design	⬜ Pending
5	Documentation	⬜ Pending
6	Project Timeline	⬜ Pending

🧭 Core Design Principles

- **Flexibility over rigidity** — almost every rule (permissions, invoice locks, partial picks, discount caps) is configurable through admin panels, not hardcoded
- **Role-based permissions as a control panel** — roles are labels; actual permissions are granular and admin-assigned per user or group
- **Mobile-first for field users** — salespeople and drivers on Android handhelds; all field-facing screens must work perfectly on mobile browser
- **Offline resilience** — system must function without live DB connection; save locally, sync on reconnect
- **Modular / optional features** — offline mode, customer portal, cloud hosting, website integration are all toggleable modules

- **Voice as a first-class input** — voice ordering matches to product catalog via cloud AI (GPT-class model), not just transcription
- **Minimize friction for field users** — smart search, favorites, recents, duplicate order — reduce scrolling and taps
- **Audit trail everywhere** — especially for invoices, payments, discounts, and any locked record modifications

---

👥 User Roles

> Roles are configurable labels. Actual permissions set per user or group in admin control panel.

Role	Primary Device	Key Responsibilities
Accounts Payable / Accounting	Desktop	Customer setup, payments, invoices, statements, notifications, transactions
Product / Inventory Admin	Desktop	Categories, subcategories, products, flavors, sizes, pricing, deposits. May sync from external system
Salesperson	Mobile Android + Desktop	Place orders, view customer account, search products, apply discounts within policy
Warehouse Worker (Picker)	Android handheld + small printer	Receive orders, pick items, mark quantities, print pick lists
Warehouse Manager	Handheld / Tablet	Approve picked orders, override picks, manage warehouse settings
Route Planner	Desktop / Tablet	Assign deliveries to routes, organize stops, manage delivery dates
Driver	Mobile Android	View route, open GPS navigation, print invoice on delivery, confirm or reject
System Admin / Owner	Desktop	All permissions, invoice lock policies, multi-admin approval workflows, global settings
Customer *(future module)*	Browser / Portal	View account, pay invoices, optionally place orders

---

🔵 PHASE 1: USER FLOW

1.1 Salesperson Order Flow

Entry points: Mobile browser (in-store, on the road) and Desktop (phone orders, office)

Order creation steps:

1. Select customer — quick search, recents list, favorites
2. Build order — multiple input methods:
   - **Voice mode** — speak naturally ("3 cases black cherry, 8 cases lemon"); AI matches to product catalog using cloud GPT-class model; plays back recording with each line highlighted for correction
   - **Browse mode** — Category → Subcategory → Product → Flavor → Size/Variant
   - **Favorites / Quick lists** — per salesperson or per customer
   - **Duplicate previous order** — reload past order and adjust quantities
3. Review full order summary
4. Submit — enters system (secondary approval optional, per permission settings)

Product catalog rules:

- Hierarchy: Category → Subcategory → Product → Flavor → Size/Variant
- Admin assigns which categories/products each salesperson can see
- Catalog layout and display order configurable globally and per salesperson
- Smart search across all levels — no endless scrolling

Scheduling:

- Orders can be placed for a future delivery date
- Default delivery date configurable per customer
- Warehouse sees upcoming orders surfaced ahead of time
- "ASAP" option for same-day or next-available delivery

1.2 Pricing & Discounts

- Base price per product (unit/case — units TBD)
- **Government deposit (CRV or equivalent)** — per container, shown separately on invoice, tracked separately in accounting
- Deposits can be discounted/waived by salesperson within policy-defined limits
- Salesperson can apply per-line-item discounts in the moment
- Discount caps enforced by policy — admin assigns policies to groups of salespeople
- Per-customer price lists — **TBD (see Open Questions)**
- Automatic promotions / volume deals — **TBD (see Open Questions)**

1.3 Warehouse Flow

- Orders transferred to warehouse on scheduled date (surfaced ahead of time)
- Picker receives order on handheld device
- Picker selects quantities per line — full, partial, or zero

- Whether partial shipments are allowed is a manager-configurable setting
- Customer charged only for what is actually picked and shipped
- Picker prints pick list to small Bluetooth printer
- Warehouse manager reviews and approves picked order before dispatch
- Manager approval required before route assignment

1.4 Delivery / Driver Flow

Route Planning (done by warehouse manager or designated planner):

- Planner sees all open orders for the day, organized by area/region
- Planner manually arranges stops in logical street order (they know the area)
- Truck loading order matters — last stop loaded first, first stop unloaded first; route order should reflect this
- Route assigned to a specific driver and truck

Driver experience:

- Driver sees their full route with stops in order on handheld device
- Opens integrated navigation per stop — Google Maps, Waze, or other (configurable)
- **Truck routing problem (flagged):** Standard GPS apps (Google Maps) don't account for truck restrictions (weight limits, low bridges, no-truck streets) — especially in dense urban areas like NYC. Possible solutions to research: use truck-specific nav (Sygic Truck, CoPilot), or note as known limitation. See Open Questions [Q24]
- Driver must follow stop order (truck loaded accordingly) but can flag issues

At each stop:

- Driver opens the order on device
- Driver can edit quantities or remove lines on-site (customer refused an item, short on truck, etc.)
- On confirmation, the system **automatically generates the invoice** from the actual delivered quantities
- Invoice is instantly posted to the system and visible to accounting in real time
- Driver then either **prints it** (Bluetooth thermal printer) **or collects a digital signature** on the device — one or the other serves as handoff proof
- Marks delivery as: **Confirmed** or **Rejected** (with reason)
- Any quantity edits are flagged to accounting and the salesperson is notified automatically
- Status updates in system in real time

Manager tracking dashboard:

- Live view of all drivers on the road — location and progress
- Per-driver: which stops completed, which pending, which rejected
- Per-order: delivered / accepted / rejected with reason and timestamp
- Rejected orders flagged immediately to sales and accounting

---

1.5 Accounting & Invoicing

- Full transaction history per customer
- Unpaid invoice tracking and aging reports
- Invoice templates — per-customer assignment, customizable fields and layout
- Invoice locking — configurable lock period after creation or payment
- Invoice lock policy set by high-admin panel; policy changes may require multi-person approval workflow
- Adjustments after lock require documented reason and appropriate permission level
- Credit memos can be issued against locked invoices (rules TBD)
- Payment processing — integrated with **Authorize.NET**
- Non-applied payment tracking and reconciliation
- Statement generation — outstanding balance, payment history, unapplied credits

Notifications:

- Fully configurable notification system — managed primarily by accounting/managers
- Channels: SMS, WhatsApp, Email
- Triggers: configurable — invoice sent, payment due, payment received, statement ready, delivery confirmed, order placed, credit limit approaching, review date reached, custom triggers
- Timing/periods: configurable — e.g. reminder 3 days before due, again on due date, again 7 days after
- Salespeople can be granted permission to enable/manage certain notifications for their own customers
- Full log of all notifications sent — who received what, when, via which channel
- Customers can optionally reply (channel-dependent — e.g. WhatsApp replies logged)

1.6 Returns & Credits

- **No reverse logistics / return workflow needed** — the company does not handle product returns in the traditional sense
- **Deposit/bottle returns** (empty containers coming back) are treated as their own product line — same as any other SKU with a deposit value attached
- Exact structure of the deposit returns product line TBD — see Open Questions [Q19]

1.7 Customer Credit Tiers & Payment Control

Customer tiers — fully configurable, admin can add/remove tiers and define rules per tier:

Tier 1 — Credit Customers (standard wholesale accounts):

- Order placed → enters warehouse flow immediately
- Pays later per payment terms (net-15, net-30, etc.)
- Credit limit set by accounting per customer

- Credit limit review is a manual scheduled feature — accountant sets a reminder/notification date (e.g. "review in 2 months"); system notifies accountant on that date to re-evaluate
- Salesperson sees warning or is hard-blocked when customer is over limit or has overdue balance (warn vs. hard-block is configurable per policy)

Tier 2 — Pay-First Customers (smaller / new / high-risk accounts):

- Order placed but held — does NOT enter warehouse flow until payment clears
- **First-time payment flow:** payment link sent to customer → customer self-pays and gives consent → card/bank info saved to vault (Authorize.NET or compatible)
- **Returning payment flow:** salesperson or accountant charges stored token directly, OR sends new payment link
- Vault options: tokenized card, direct payment info (for legacy payment systems) — configurable
- Only after payment confirmed does order release to warehouse

Credit tier system design:

- Admin can create any number of tiers with custom names and rules
- Rules per tier are configurable: payment timing, credit limits, block vs. warn behavior, notification triggers, discount caps, etc.
- Assign customers to tiers individually or by group/policy
- All credit decisions and tier changes logged with full audit trail

---

1.9 Multi-Location Customers

- A single customer (company/account) can have multiple delivery locations (store branches)
- Each location has its own delivery address, contact, and potentially its own order history
- Billing/accounting rolls up to the parent account level
- Credit limit, payment terms, tier — set at parent account level
- Salesperson selects customer → then selects which location when placing an order
- Exact account hierarchy structure TBD — see Open Questions [Q25]

1.10 User Onboarding

- Formal in-app onboarding flow TBD — depends on client preference
- Options: guided in-app setup wizard, or informal (admin sets up users directly, trains in person)
- See Open Questions [Q26]

1.8 Salesperson Dashboard & Reports

Default view — "smart summary" between simple and informative:

- Customer list with at-a-glance status: last order date, outstanding balance, credit available, overdue flag
- Quick actions: place new order, view account, call/message customer
- Upcoming deliveries for their accounts
- Recent activity feed

Customizable views:

- Each salesperson configures their own default dashboard — what widgets, what columns, what order
- Save frequently-used reports as quick-access shortcuts
- Admin can set a company-wide default view that applies until salesperson customizes it

Reports & dashboards:

- Modern, smart, visual — charts, summaries, trends
- Per-customer: full transaction history, invoices, payments, aging, balance, credit status
- Portfolio view: all their customers in one list — who owes what, aging buckets, who to follow up with
- Sales performance: revenue by period, top products, top customers
- Custom report builder — salesperson defines columns, filters, date ranges, saves as named report

Admin back-end (separate from field UI):

- Power-user control panel for managers/admins who want deep configurability
- Manage: roles, permissions, policies, groups, discount caps, credit tiers, invoice lock rules, notification rules, product catalog structure, etc.
- Clearly separated from the clean field-facing UI so it doesn't clutter the day-to-day experience
- Think of it as two apps in one: a clean simple front-end for field users, a powerful back-end for admins

---

1.11 Product Catalog Management

Admin/manager panel:

- Separate back-end panel for managing what salespeople see and how
- Create/edit: categories, subcategories, products, flavors, sizes, variants
- Set pricing, deposit values, tax flags per product
- Control visibility: assign categories/products to specific salespeople or groups
- Set display order and layout of catalog
- Enable/disable products (out of stock, seasonal, discontinued)

Salesperson catalog view:

- Salespeople can customize how they personally see and navigate the catalog
- Save favorite products, custom quick-lists per customer
- View preferences (grid vs. list, sort order, etc.) saved per user
- Company default view set by admin, overridable by salesperson

Inventory integration:

- Full details TBD pending stakeholder answer on current inventory/manufacturing system
- Goal: pull live inventory data automatically where possible
- Someone should still have manual override ability
- Out-of-stock handling during order flow TBD — see Open Questions [Q1, Q27]

> Inventory system details pending — see Open Questions [Q1] [Q27]

1.12 Statement Generation

- Default: monthly auto-send per customer
- Fully flexible — configurable per customer or per group/policy
- Options: frequency (weekly, monthly, custom), delivery channel (email, WhatsApp, SMS), auto vs. manual
- Manual send always available — accountant can send on demand at any time
- Statement content: outstanding invoices, payment history, unapplied credits, balance due
- Different customers/groups can have different statement schedules

---

1.13 New Customer Onboarding Flow

- Standard fields: company name, main contact, contact persons, email addresses, phone numbers
- Delivery address(es) — supports multiple locations under one account
- Billing address (may differ from delivery)
- Payment method(s) — card on file / vault token / payment terms
- Credit tier assignment
- Assigned salesperson(s)
- Any required documents or approvals TBD with accountant
- Admin or accounting creates the account; salesperson may be granted permission to create with limited fields

---

1.14 Order Modifications, Cancellations & Adjustments

- Salesperson can edit a placed order freely up to a configurable cutoff point (e.g. before warehouse starts picking)
- After picking begins — modifications restricted; who can do what is policy-controlled
- Cancellations and post-pick adjustments: rules TBD with stakeholders — will have a control panel similar to discount caps
- Adjustment caps: admin sets per-role limits on what adjustments salespeople or accountants can make
- All adjustments require a documented reason and are fully audit-logged

- Goal: more controlled than QuickBooks (which is too open) but not completely locked down
- ⬜ Exact rules TBD — see Open Questions [Q28, Q29]

1.15 Manager / Owner Dashboard & Reports

- Current state: legacy local system accessed via RDP, limited reporting, mostly raw tables
- New system should be a significant upgrade — modern, visual, intuitive
- **Standard reports to include (baseline):** sales by period, sales by salesperson, sales by customer, sales by product/category, outstanding receivables, aging report, payment collection rate, delivery performance (on-time, rejected), top customers, top products
- Reports should be visual — charts, trend lines, summaries, not just tables
- Custom report builder — filter by any field, date range, group by any dimension, save as named report
- Manager can see company-wide view; salesperson sees only their accounts
- All reports configurable and savable — manager sets up their own dashboard
- Feature ideas to propose to client: KPI tiles (revenue today/week/month vs. prior period), route completion live tracker, cash flow forecast, customer health score (order frequency + payment behavior)
- Client will review a summary before development begins — they can request additions or removals

---

🟡 Optional Feature Modules

Module	Description	Priority
Basic Offline Mode	Orders + delivery saved on device, sync on reconnect, conflict resolution	**INCLUDED IN BASE**
Temp Cloud Backup	Cloud fallback during server downtime	Optional
Digital Pick on Handheld	Picker uses tablet/handheld, manager approves digitally	Optional
AI Pick Sheet OCR	Photo scan of paper pick sheet, AI reads handwriting	Optional
Voice Ordering	Speak or upload recording/voice note, AI matches to catalog	Optional
SMS Notifications (Twilio)	SMS channel for notifications	Optional
WhatsApp Notifications	WhatsApp channel for notifications	Optional
Credit Scoring System	Auto credit rating based on usage/payments, auto-block, flag	Optional
Customer Portal	Customers view accounts, pay invoices, optionally place orders	Optional

Module	Description	Priority
Customer-Facing Website	For smaller customers — orders and account view	Optional
Multi-Company / Multi-Tenant	Each customer as separate accounting entity	Optional
Full Private Network Lockdown	VPN-only access, private DNS, zero public exposure	Optional
SSO + MFA Integration	Azure AD / LDAP single sign-on with multi-factor auth	Optional
Cloud App Deployment	App on cloud, DB stays on-prem initially	Future
Extended Support (3 months)	Bug fixes, minor adjustments, monitoring post-launch	Optional
Data Migration	Import from legacy system	Optional
Truck-Specific Navigation	Sygic/CoPilot integration for truck restrictions	Optional
AI-Powered Reports	Plain English → SQL/Python queries. AI sees metadata only, never data. Read-only execution. Server sandbox for Python scripts	Optional

---

🚩 Open Questions — Stakeholder Interview Required

- **[Q1]** What inventory/manufacturing system is currently in use? Does it have an API or data export?
- **[Q2]** Who is the owner-level admin — one person or a small group?
- **[Q3]** Is there a credit approval step when onboarding a new customer?
- **[Q4]** Are discounts per product, per customer, per order — or all three?
- **[Q5]** What is the current invoicing system? What data needs to migrate?
- **[Q6]** Voice recognition: on-device (offline) or cloud-only? — *NOTED: Cloud preferred*
- **[Q7]** Should the voice system recognize product SKU codes in addition to names?
- **[Q8]** When duplicating a past order, should system check current inventory availability?
- **[Q9]** Are deposits (CRV) per unit or per case? Do they vary by container type (can vs bottle vs size)?
- **[Q10]** Do some customers have fully custom price lists, or always standard price minus a discount?
- **[Q11]** Are there automatic promotions or volume deals (buy X get Y, tiered pricing)?
- **[Q12]** Is tax calculated on the deposit separately from the product price?
- **[Q13]** How are returns/credits handled? Does the driver pick up returns on delivery?
- **[Q14]** Who can approve a return or issue a credit memo?

- **[Q15]** What is the exact invoice lock period the company wants to start with?
- **[Q16]** How many salespeople, warehouse workers, and drivers are there approximately?
- **[Q17]** What Bluetooth printer models are currently in use or planned?
- **[Q18]** Is there an existing customer database that needs to be imported?
- **[Q19]** How exactly are deposit/bottle returns handled as a product line? Are they a negative line item on the next order, a separate credit transaction, or something else?
- **[Q20]** Confirm with accountant: how many default credit tiers to start with, and what are their names/rules?
- **[Q21]** What payment vault/legacy systems need to be supported beyond Authorize.NET?
- **[Q22]** Should the credit review notification go to the accountant only, or also CC the salesperson on that account?
- **[Q23]** What default dashboard view does management want for salespeople out of the box?
- **[Q24]** Truck routing: should the app integrate a truck-specific navigation solution, or is standard GPS (Google Maps/Waze) acceptable with drivers using their own judgment?
- **[Q25]** Multi-location customers: confirm account hierarchy — parent account with child locations? Any cases where a location has its own separate billing?
- **[Q26]** User onboarding: invest in guided in-app onboarding, or handle training in person with simple admin setup?
- **[Q27]** When a product is out of stock between order placement and warehouse picking, what is the rule — auto-remove from order, flag to salesperson, or let warehouse decide? Are they a negative line item on the next order, a separate credit transaction, or something else?
- **[Q32]** ~~Order vs Invoice: confirm that orders and invoices are separate records (order is placed → invoice is generated from it once delivered). Or should the order become the invoice?~~ ✅ **ANSWERED:** Separate records. Invoice is auto-generated at the moment the driver confirms delivery, from the actual delivered quantities. Posted to system immediately.
- **[Q33]** End-of-week invoice finalization: accounting mentioned they want to review and confirm/finalize all invoices before they are "posted" at end of week. What exactly does "posted" mean in your workflow — locked into the ledger? Sent to customer? Both? What specific discrepancies should AI flag during this review (pricing errors, unusual quantities, missing deposits, duplicate charges)?
- **[Q34]** Customer classification/tags: do you want to tag/categorize customers by type (e.g. restaurant, grocery, gas station, chain, independent)? Would this affect pricing, catalog visibility, or reporting?

⏭️ Next Steps

1. **Continue now:** Phase 2 — UI / App Design
2. **Parallel:** Stakeholder call using Stakeholder_Questions.md
3. **After answers:** Update open questions and adjust design decisions accordingly
4. **Then:** Remaining accounting edge cases
5. **Then:** Warehouse/delivery edge cases

6. **Stakeholder interview:** Use Open Questions list above
7. **Session 2:** App Design — screens, navigation, UX structure

---

🟡 PHASE 2: UI / APP DESIGN

2.1 Platform & Technology Direction

- **Platform:** Web browser-based — works on desktop, tablet, and mobile (responsive)
- **Deployment:** Installed on client's Windows server; web stack served locally (on-prem)
- Server setup: will need Node.js / web server packages installed on Windows — exact specs TBD pending IT conversation
- Mobile users (salespeople, drivers, warehouse) access via mobile browser — no native app needed
- Offline capability handled at the web app layer (service workers, local storage, sync)

2.2 UI Design Principles

The brief (directly from client):

- Not fancy — decent, clean, modern. Not Excel-with-buttons.
- Fast and responsive — snappy, no lag, no waiting
- Not overcomplicated — clean default views, complexity hidden until needed
- Settings/control panels are powerful but kept *separate* from the main workflow
- Smart and adaptive — UI adjusts based on context, role, and user preferences
- Lots of options, but surfaced intelligently — not dumped on the user at once

Design language:

- Clean, minimal chrome — content-first, not UI-first
- Modern data display — cards, summary tiles, smart tables (sortable, filterable, not static grids)
- Clear visual hierarchy — most important info largest, secondary info smaller
- Color used for status and alerts, not decoration (e.g. red = overdue, green = paid, yellow = pending)
- Consistent action patterns — same gestures/clicks do the same things everywhere
- Mobile: touch-friendly, large tap targets, minimal typing where possible

Two-layer UI model:

- **Layer 1 — Field UI:** Clean, focused, fast. What salespeople, drivers, and warehouse workers see. No clutter.
- **Layer 2 — Admin/Back-end:** Power settings, control panels, policies, configurations. Separate area, separate mindset.
- Users only see what their role needs — complexity hidden by default, accessible when needed

2.3 Screen Inventory

Planned screen groups:

- Auth: Login, role-based redirect on login
- Salesperson: Dashboard, Customer List, Customer Profile, New Order, Order Review, Order History
- Warehouse: Order Queue, Pick Order, Pick Confirmation, Print Management
- Driver: My Route, Stop Detail, Delivery Confirmation, Navigation Launch
- Accounting: Dashboard, Customer Account, Invoice Detail, Payment Processing, Statements, Notifications
- Route Planner: Open Orders, Route Builder, Route Assignment
- Admin Back-end: User Management, Roles & Permissions, Product Catalog, Pricing & Policies, Invoice Rules, Notification Rules, Credit Tier Settings, Discount & Adjustment Caps
- Manager: Reports Dashboard, Live Route Tracker, Sales Reports, Receivables
- Reports: Report Builder, Standard Reports Library, Saved Reports, Scheduled Exports, AI Query Interface (optional)

2.4 Salesperson Screens — Detailed Flow

2.4.1 Login & Role-Based Redirect

- User logs in → system detects role → redirects to role-appropriate dashboard
- Salesperson → Salesperson Dashboard
- Driver → My Route screen
- Warehouse → Order Queue
- Accounting → Accounting Dashboard
- Admin → Admin Control Panel

2.4.2 Salesperson Dashboard (Hub)

- **Minimal hub** — not a heavy analytics page, more of a launchpad with at-a-glance info
- **Quick-action buttons:** New Order, Customer List, My Orders Today, Messages/Alerts
- **Summary tiles:** Orders today, outstanding balance total, deliveries in progress, alerts/flags
- **Recent activity feed** — last few orders, recent payments, notifications (compact)
- **Link to "Reports" page** — separate detailed reports screen where salesperson can build, save, and access custom report views (not cluttering the dashboard)

2.4.3 Customer List

- Default: list view with key info visible per row — customer name, last order date, outstanding balance, overdue flag, credit status
- Toggle option: switch to card view for a more visual layout
- **Search & filter bar** at top — search by name, filter by status, area, tier, etc.
- **"New Order" button directly on each row** — salesperson can start an order without entering the customer profile first

- Click customer name → opens Customer Profile

2.4.4 Customer Profile

Four tabs:

- **Overview** — contact info, delivery addresses, billing address, credit status, tier, payment terms, assigned salesperson
- **Orders** — order history with status (placed, picking, shipped, delivered, rejected). Can duplicate any past order. Can view order detail. Filter by date range, status
- **Account** — invoices, payments, balance, aging buckets, statements, unapplied credits. Links to pay/send statement
- **Notes / Activity** — internal notes (salesperson can add), full notification log (what was sent when), activity feed (orders, payments, changes)

2.4.5 New Order — Step-by-Step Wizard

Step 1: Select Customer

- Quick search, recents, favorites
- If started from customer list "New Order" button → this step is pre-filled, skip to Step 2
- Select delivery location if multi-location customer

Step 2: Build Order — Add Items

- Multiple input modes available:
  - **Browse:** Category → Subcategory → Product → Flavor → Size — add quantity
  - **Search:** Smart search bar — type product name, SKU, or partial match
  - **Voice:** Tap mic, speak naturally, AI matches to catalog, review matches
  - **Favorites / Quick List:** Per-customer or per-salesperson saved product lists
  - **Duplicate Past Order:** Select a previous order → loads all items with quantities → edit as needed
- Running item count / subtotal visible at top or bottom of screen
- Each added item shows: product name, qty, unit price, line total, discount (if any)
- Can adjust quantity or remove items inline

Step 3: Review Order (Editable Invoice Preview)

- Looks like the final invoice layout — but every field is still editable
- Line items with product, qty, unit price, discount, deposit, line total
- Subtotal, deposits total, tax, grand total
- Delivery date selector (default pre-filled per customer settings, editable)
- Notes / special instructions field
- Salesperson can still add/remove/edit items at this stage
- Discount per line or per order — within policy caps (system enforces)

Step 4: Submit

- Confirm and submit → order enters the system
- If customer is Tier 2 (pay-first) → system holds order and triggers payment flow
- If secondary approval required (per policy) → order goes to approval queue
- Confirmation screen with order number, summary, estimated delivery date

---

2.5 Product Display Rules

- **All products must have a 5-digit numeric-only code** (e.g. 10023) — this is the universal product identifier across the system
- **Auto-generated** when product is first added, with structured hierarchy (encodes category → subcategory → type → flavor where practical)
- **Catalog view differs by role — customizable:**
  - **Salesperson view:** consumer-oriented units — e.g. "10 × 12-packs"
  - **Warehouse worker view:** warehouse-oriented units — e.g. "1 pallet" or "10 cases"
- Display unit mapping is configurable per product in admin — defines how the same quantity is described to different roles
- The underlying data is always the same; only the display label changes per role context

---

2.6 Warehouse Screens — Detailed Flow

2.6.1 Warehouse Order Queue

- **Grouped by route / delivery area** — picker sees orders organized by which truck/route they belong to
- Orders show: customer name, product code, item count (in cases), delivery date, route assignment
- **Manager assigns orders to specific pickers** — pickers don't self-claim
- Filter/sort by: date, route, priority, status
- Status columns within each group: **New** → **Assigned** → **Picked** → **Approved**
- Manager must approve picked orders before they move forward

2.6.2 Pick Flow — Three Tiers

BASIC (included): Paper Pick + Manual Entry

- Manager prints pick list for the order
- Worker picks using pen and physical clipboard — marks quantities by hand on the printout
- Worker returns completed clipboard to manager
- Manager manually enters results in the system: removes lines or edits quantities, rest is confirmed
- Confirmed pick → order moves to approved status

OPTIONAL: Digital Pick (picker on handheld/tablet)

- Picker opens assigned order on device
- Sees line items: 5-digit product code, product name, requested qty (in cases/pallets)
- Enters picked qty per item — can be less than requested (partial pick if allowed by policy)
- Submits pick → goes to manager for approval

OPTIONAL: Print & Scan (AI OCR)

- Worker picks using pen and physical clipboard — marks quantities by hand on the printout
- Manager snaps a photo of the completed pick sheet
- AI analyzes the photo — extracts handwritten quantities, maps to line items using the 5-digit product codes
- Manager sees AI-extracted data on screen → reviews and confirms (can correct any AI misreads)
- Confirmed pick → order moves to approved status

> All modes result in the same data: a confirmed pick with actual quantities per line item. Mode selection is configurable per warehouse or per manager preference.

2.6.3 Pick Approval

- Manager reviews all picked orders before they are released
- Can override quantities, flag issues, or reject back to picker
- Approved orders → ready for route assignment and truck loading

---

2.7 Route Planner Screens — Detailed Flow

> **Important:** Route planning happens BEFORE the truck is loaded. The route determines the loading order (last stop loaded first).

2.7.1 Route Builder

- Planner sees all **approved/picked orders** for a given delivery date
- Orders displayed grouped by area/region — list view + optional map view for geographic context
- Planner drags orders into **route groups** — each route = one truck + one driver
- Within each route, planner arranges **stop order** (they know the streets/area)
- System shows: total cases/weight per route, number of stops, estimated time
- Assign route to a specific driver and truck
- Once finalized → route is locked and sent to warehouse for truck loading (load in reverse stop order)

2.7.2 Route Assignment View

- Summary of all routes for the day: driver, truck, # stops, # cases, status
- Can reassign driver/truck, reorder stops, add/remove orders until truck loading begins

2.8 Driver Screens — Detailed Flow

2.8.1 My Route

- Driver opens app → sees their assigned route for the day
- List of stops in delivery order: stop #, customer name, address, # cases, delivery window
- Overall progress bar: X of Y stops completed
- Tap a stop → opens Stop Detail

2.8.2 Stop Detail

- Customer name, address, contact phone
- Order summary: line items with 5-digit codes, product names, quantities (cases)
- **Driver can view and EDIT the order before confirming** — e.g. customer spots a discrepancy at the door. Edits flag for accounting review and notify the salesperson automatically
- **Print Invoice button** — prints to Bluetooth printer
  - ⚠️ **Location check:** If driver is NOT near the delivery address when they tap Print → **medium warning** ("You don't appear to be at this location. Print anyway?") — not a hard block, but logged
- **Digital Signature option** — customer can sign on device screen instead of receiving printout
- **Launch Navigation** — opens Google Maps / Waze / configured nav app with the stop address
- **Delivery Confirmation:**
  - **Confirmed** — delivery accepted, tap to confirm
  - **Rejected** — select reason from dropdown (not home, refused, damaged, wrong order, other) + optional note
  - Status update pushes to system in real time

2.8.3 End of Route

- Summary: stops completed, stops rejected (with reasons), total delivered
- Driver confirms route complete → data finalized

2.9 Auto-Delivery Confirmation Rule

> **Key rule:** When the driver **prints the invoice** (Bluetooth) or **sends it electronically** (email/WhatsApp/SMS) at a stop, the delivery status **automatically changes to "Delivered/Confirmed"** — no extra tap needed. The driver only needs to interact if they are **rejecting** the delivery (must select reason + optional note). This removes friction from the happy path.

2.10 Order Modification & Adjustment Policy

- **Before picking starts:** Salespeople and accounting can apply all kinds of changes — add/remove items, adjust quantities, apply discounts, change delivery date, cancel order. Full flexibility.
- **After picking begins:** Modifications become **very strict** — limited adjustments only, controlled by configurable policies per role, per user, and per group
- **Policy engine:** Admin defines what adjustments are possible post-pick, who can make them, and what caps/limits apply. Sensible defaults out of the box, fully adjustable.
- **Discount application:** Salespeople and accounting can apply discounts freely until the order is picked. Post-pick discount adjustments follow the same strict policy controls.
- **Audit trail:** Every change at every stage is logged — who, what, when, reason

---

2.11 Accounting Screens — Detailed Flow

> **Design principle:** Simpler than heavy accounting apps (not QuickBooks, not SAP). Clean and approachable UI. BUT — under the hood, the system follows **proper accounting principles**: double-entry ledger, journal entries, chart of accounts, transaction records, and standard financial statements. The user doesn't need to be an accountant to use it — the system handles the accounting mechanics automatically when they perform business actions (invoice, payment, credit, adjustment).

2.11.1 Accounting Fundamentals Built Into the System

Double-Entry Ledger (runs automatically behind every action):

- Every financial event creates proper journal entries — invoice created, payment received, credit issued, discount applied, deposit charged, adjustment made
- Users don't manually create journal entries — the system generates them from business actions
- But an accountant CAN view the full ledger and drill into any entry for audit/verification

Chart of Accounts (pre-configured, admin-adjustable):

Account	Type	Purpose
Accounts Receivable	Asset	What customers owe
Sales Revenue	Revenue	Product sales income
Deposit Revenue (CRV)	Revenue/Liability	Government deposits collected
Deposit Returns	Contra-Revenue	Deposits refunded on returns
Discounts Given	Contra-Revenue	All discounts applied
Cash / Bank	Asset	Money received
Unapplied Payments	Liability	Payments received but not matched to invoices
Credit Memos	Liability	Credits issued to customers

Account	Type	Purpose
Sales Tax Payable	Liability	Tax collected, owed to government

> Admin can add/rename accounts as needed. System enforces that debits = credits on every transaction.

What happens automatically:

Business Action	System Creates
Invoice generated	Debit: Accounts Receivable → Credit: Sales Revenue + Deposit Revenue + Tax Payable
Payment received	Debit: Cash/Bank → Credit: Accounts Receivable
Discount applied	Debit: Discounts Given → Credit: Accounts Receivable
Credit memo issued	Debit: Credit Memos → Credit: Accounts Receivable
Unapplied payment	Debit: Cash/Bank → Credit: Unapplied Payments (until matched)

Standard Financial Reports (auto-generated):

- **Accounts Receivable Aging** — who owes what, how overdue (current, 30, 60, 90+ days)
- **Customer Statement** — per-customer: invoices, payments, credits, balance
- **Transaction Journal** — chronological log of all financial entries
- **Revenue Report** — sales by period, by product, by customer, by salesperson
- **Cash Receipts Report** — payments received by period, by method
- **Tax Report** — tax collected by period (for government filing)
- **Trial Balance** — all account balances at a point in time (for accountant verification)

2.11.2 Accounting Dashboard

- **KPI tiles:** Total receivables, overdue amount, payments received today/this week, cash flow summary
- **Action queue:** Invoices pending review, unapplied payments to reconcile, credit limit reviews due, flagged orders (rejected deliveries, over-limit customers)
- **Quick access:** Customer lookup, send statement, process payment, issue credit memo
- **Today's deliveries/orders visible** — accounting needs to see what's going out (for invoicing awareness)

2.11.3 Invoice Detail Screen

- Full invoice view — same layout as printed/sent invoice
- Payment status and payment history for this invoice
- Actions: apply payment, issue credit memo, send to customer (email/WhatsApp/SMS), print, adjust (if within lock policy)
- **PDF preview and download** — for email attachment or file storage

- Linked journal entries visible (expandable) — for accountant who wants to see the ledger impact

2.11.4 Payment Processing

- **Both entry points available:**
  - From customer → see all open invoices → select which to pay → enter amount → process via Authorize.NET
  - From invoice → click "Apply Payment" → enter amount → process
- **Batch payment view** — select multiple invoices across multiple customers, process in one session
- **Unapplied payment handling:** If payment doesn't match exact invoice total → remainder goes to Unapplied Payments → accountant matches later
- **Payment methods:** Authorize.NET (card), ACH/bank, check (manual entry), cash (manual entry)

2.11.5 Statements

- Select customer → generate statement (outstanding invoices, payments applied, credits, running balance)
- Send via email / WhatsApp / SMS per customer preference
- **Batch statement run** — generate and send statements for all customers (or filtered group) at once
- Configurable schedule — monthly, weekly, custom per customer or group
- Auto-send on schedule + manual send on demand anytime

2.12 Admin / Back-End Screens

> **Separate area from field UI.** Power-user control panel. Only accessible to users with admin permissions.

2.12.1 Admin Panel — Main Sections

1. **Users & Roles** — create/edit users, assign roles, set granular permissions per user or group, manage teams
2. **Product Catalog** — categories, subcategories, products, 6-digit codes, pricing, deposits, display units (sales vs. warehouse), visibility rules per salesperson/group
3. **Policies** — discount caps (per role/user/group), adjustment rules (pre-pick open, post-pick strict), invoice lock period, order modification rules, credit tier definitions, approval workflows
4. **Notifications** — define triggers (invoice sent, payment due, delivery confirmed, etc.), channels (email/SMS/WhatsApp), timing/frequency, templates per event type
5. **System Settings** — company info, default delivery date rules, payment gateway config (Authorize.NET), Bluetooth printer settings, statement schedules, location warning thresholds

2.12.2 Each admin section has:

- List view with search/filter
- Detail/edit form per item
- Audit log — who changed what, when

2.13 Manager / Owner Dashboard

- **All features included by default** — owner sees everything, can customize which sections/tiles are visible
- **Multiple saved views** — manager can create and switch between different dashboard layouts (e.g. "Daily Overview", "Sales Focus", "Collections Focus")

Default view includes:

- **Live Route Tracker** — all drivers on a map in real time, per-driver stop progress, color-coded status
- **KPI Tiles:** Revenue today/week/month vs prior period, total deliveries, rejection rate, collection rate, outstanding receivables
- **Salesperson Performance** — individual and comparison/leaderboard view, revenue, orders, new customers, collection rate
- **Reports Section:** Sales by salesperson, by customer, by product/category, aging report, receivables, payment history — all with charts and trend lines
- **Custom Report Builder** — pick columns, filters, date ranges, group-by dimensions, save as named report, schedule auto-generation
- **Alerts & Flags** — rejected deliveries, over-limit customers, overdue accounts, pending approvals

View customization:

- Drag-and-drop tile arrangement
- Show/hide any section
- Save multiple named views, switch between them
- Admin can set a company-wide default view for managers; each manager can override with their own

---

2.14 Reports — Detailed Flow

> **Design principle:** Reports are a first-class feature, not an afterthought. Every user should be able to get answers from system data without asking IT or exporting to Excel.

2.14.1 Report Builder (Base — All Roles)

- **User-friendly interface** — no SQL knowledge needed, no technical jargon
- **Build a report in 3 steps:**
  1. **Pick data source(s)** — Orders, Invoices, Payments, Customers, Products, Deliveries, Routes, etc.
  2. **Select columns** — drag/drop or checkbox to pick fields. System automatically handles joins when combining data from multiple sources (e.g. Orders + Customers + Products)
  3. **Filter & sort** — date range, status, customer, salesperson, area, product category, any field. Multiple filters combinable with AND/OR
- **Live preview** — results update as filters are changed

- **Simple graphs auto-generated** — when the report includes numeric/metric columns (revenue, quantities, counts), system automatically offers chart options:
  - Bar chart, line chart, pie chart — pick one or auto-suggested based on data shape
  - Charts are interactive — hover for values, click to drill down
- **Export options:** PDF, Excel (.xlsx), CSV
- **Grouping & aggregation** — group by any dimension (customer, product, salesperson, date period), auto-calculate sums, averages, counts per group

2.14.2 Standard Pre-Built Reports (Base)

Shipped with the system — ready to use out of the box, customizable:

Sales Reports:

- Sales by period (daily, weekly, monthly, custom range)
- Sales by salesperson (with comparison/leaderboard)
- Sales by customer (top customers, revenue ranking)
- Sales by product / category / subcategory
- Sales by area / zip code / customer group
- Order volume trends (line chart over time)

Accounting Reports:

- Accounts Receivable Aging (current, 30, 60, 90+ days)
- Customer statements (outstanding invoices, payments, credits, balance)
- Revenue report by period
- Cash receipts report (payments by method, by period)
- Tax report (for government filing)
- Trial balance
- Transaction journal (chronological ledger)
- Unapplied payments report
- Credit memo report

Operations Reports:

- Delivery performance (on-time rate, rejection rate, reasons)
- Route efficiency (stops per route, cases per route)
- Warehouse pick accuracy
- Driver performance
- Inventory movement

Customer Reports:

- Customer activity (orders, payments, credits over time)

- New customer acquisition by period
- Customer retention / churn
- Credit utilization

2.14.3 Saved Reports & Scheduled Exports (Base)

- **Save any report** — give it a name, save filters/columns/sort/chart settings
- **Recurring exports** — schedule a saved report to auto-run and export (e.g. "Sales by Salesperson — Weekly" auto-generates every Monday and emails to manager)
- **Report library** — each user has their saved reports; admin can publish company-wide shared reports
- **Quick-access dashboard** — pin favorite reports to dashboard for one-click access

2.14.4 Role-Based Report Access

- Salespeople see only reports scoped to their assigned customers
- Accounting sees all financial data
- Managers see everything
- Admin controls who can access which report types and data scopes via the same permission engine

2.14.5 AI-Powered Reports (OPTIONAL MODULE)

> **CRITICAL PRIVACY GUARANTEE:** The AI model **NEVER sees actual business data**. It only receives **metadata** — table names, column headers, column data types, and relationships. The AI writes queries; it never reads results.

How it works — step by step:

1. User types a question in plain English: "*Show me total revenue by salesperson for the last 3 months, compared to the previous 3 months*"
2. System sends **only metadata** to the AI model:
   - Table names: `orders`, `invoices`, `users`
   - Column headers: `invoice_date`, `grand_total`, `salesperson_id`, `full_name`
   - Column types: `date`, `decimal`, `uuid`, `string`
   - Table relationships: `invoices.salesperson_id → users.user_id`
   - **NO actual data values are ever sent**
3. AI writes a **read-only SQL query** based on the metadata
4. Query is executed on the server against the database in **read-only mode** (SELECT only — no INSERT, UPDATE, DELETE, DROP, or any write operation possible)
5. Results are displayed to the user in the report builder UI with auto-generated charts
6. **Query results are NEVER sent back to the AI** — the AI's job is done after writing the query

Advanced mode — Python scripts (OPTIONAL, part of AI module):

- For complex analysis that SQL alone can't handle (statistical analysis, forecasting, custom calculations)
- AI writes a Python script based on the same metadata-only approach
- Script executes in a **sandboxed environment on the server** — isolated process, no network access, no file system access outside sandbox, memory-limited, time-limited
- Script can only read data (read-only DB connection), perform calculations, and output results
- Results displayed in the report builder UI
- Scripts are logged and can be reviewed by admin

Security layers:

Layer	Protection
AI input	Metadata only — table names, column headers, types. Zero data values
AI output	SQL query or Python script text only
Query execution	Read-only database connection (SELECT only)
Data return	Results go to user's screen only. Never sent to AI
Python sandbox	Isolated process, no network, no filesystem, memory cap, time cap
Audit	Every AI-generated query logged with user, timestamp, query text

---

🔴 PHASE 3: DATA MODEL

> **Purpose:** Define the core entities, their fields, and relationships. This is the blueprint for the database and drives everything — UI, API, accounting, reports.

3.1 Customer

Parent Account (Company Level):

Field	Type	Notes
customer_id	UUID	Primary key
company_name	string	Required
billing_address	Address object	Street, city, state, zip
credit_tier_id	FK → CreditTier	Tier 1 (credit), Tier 2 (pay-first), custom
credit_limit	decimal	Set by accounting
payment_terms	enum	net-15, net-30, COD, custom

Field	Type	Notes
tax_exempt	boolean	Default false
assigned_salesperson_ids	FK[] → User	One or more salespeople
default_statement_schedule	config object	Frequency, channel, auto vs manual
payment_vault_token	encrypted string	Authorize.NET token
outstanding_balance	decimal (computed)	Sum of unpaid invoices
unapplied_credits	decimal (computed)	Payments not yet matched
status	enum	active, inactive, suspended
notes	text	Internal notes
created_at	timestamp	
created_by	FK → User	
tags	string[]	Optional: restaurant, grocery, chain, etc.

Child Location (Delivery Site):

Field	Type	Notes
location_id	UUID	Primary key
parent_customer_id	FK → Customer	Links to parent account
location_name	string	e.g. "Store #3 - Brooklyn"
delivery_address	Address object	
contact_person	string	
phone	string	
email	string	
default_delivery_day	enum	Mon-Sun or null
delivery_time_window	string	e.g. "6am-10am"
delivery_notes	text	Gate code, dock instructions, etc.

3.2 Product & Catalog

Product Hierarchy: Category → Subcategory → Product → Flavor → Size/Variant

Category:

Field	Type	Notes
category_id	UUID	
name	string	e.g. "Beverages", "Snacks"
display_order	int	Sort position in catalog
status	enum	active / inactive

Subcategory:

Field	Type	Notes
subcategory_id	UUID	
category_id	FK → Category	
name	string	e.g. "Sparkling Water", "Juice"
display_order	int	
status	enum	active / inactive

Product:

Field	Type	Notes
product_id	UUID	Internal PK
product_code	char(6)	**6-digit numeric code** — unique, user-facing identifier
name	string	e.g. "Crystal Sparkling Water"
description	text	Optional
subcategory_id	FK → Subcategory	
status	enum	active / inactive / discontinued
image_url	string	Optional product image
created_at	timestamp	

Product Variant (Flavor + Size combination):

Field	Type	Notes
variant_id	UUID	
product_id	FK → Product	
variant_code	char(6)	**6-digit code** — each sellable variant has its own code

Field	Type	Notes
flavor	string	e.g. "Black Cherry", "Lemon" (nullable if N/A)
size	string	e.g. "12oz", "1L", "2L"
base_price	decimal	Price per selling unit
deposit_amount	decimal	CRV/deposit per unit
tax_rule	enum	taxable / exempt / deposit-only-taxable
status	enum	active / inactive

Packaging / Display Units (flexible, per variant):

Field	Type	Notes
packaging_id	UUID	
variant_id	FK → ProductVariant	
unit_name	string	e.g. "12-pack", "case", "pallet"
units_per_package	int	e.g. 12, 24, 48
display_context	enum	salesperson / warehouse / both
is_default	boolean	Default display for this context

> **Flexibility:** A single variant (e.g. 12oz Black Cherry) can have multiple packaging definitions — a salesperson sees "12-pack" while warehouse sees "case of 2 trays". Admin defines as many packaging levels as needed per product. Products can be combined into bundles/kits for different customer levels.

Catalog Visibility (who sees what):

Field	Type	Notes
visibility_id	UUID	
entity_type	enum	category / subcategory / product / variant
entity_id	UUID	FK to the relevant catalog entity
visible_to_type	enum	user / group / role / all
visible_to_id	UUID	FK to user/group/role (null if "all")

3.3 Order

> **Orders and invoices are separate records.** An order tracks what was requested; an invoice is generated from the delivered/picked quantities. *(Pending stakeholder confirmation — see Q32)*

Order Header:

Field	Type	Notes
order_id	UUID	
order_number	string	Auto-generated, human-readable (e.g. ORD-2026-00142)
customer_id	FK → Customer	
location_id	FK → Location	Delivery site
salesperson_id	FK → User	Who placed it
status	enum	draft / placed / approved / picking / picked / routed / loaded / in-transit / delivered / rejected / cancelled
requested_delivery_date	date	
actual_delivery_date	date	Filled on delivery
notes	text	Special instructions
created_at	timestamp	
modified_at	timestamp	
modification_log	JSON[]	Array of change records with who/what/when/reason

Order Line Item:

Field	Type	Notes
line_id	UUID	
order_id	FK → Order	
variant_id	FK → ProductVariant	
product_code	char(6)	Denormalized for display/search
product_name	string	Denormalized — snapshot at time of order
ordered_qty	int	What salesperson requested (in selling units)
picked_qty	int	What warehouse actually picked (null until picked)

Field	Type	Notes
unit_price	decimal	Price at time of order (snapshot)
discount_amount	decimal	Per-line discount applied
discount_reason	string	Optional
deposit_per_unit	decimal	CRV at time of order
line_total	decimal (computed)	(picked_qty or ordered_qty) × (unit_price - discount) + deposits
tax_amount	decimal (computed)	

3.4 Invoice

> Generated from a delivered order. Represents the financial record. *(Pending: exact timing of generation — on delivery vs. end-of-week finalization — see Q33)*

Invoice Header:

Field	Type	Notes
invoice_id	UUID	
invoice_number	string	Auto-generated (e.g. INV-2026-00142)
order_id	FK → Order	Source order
customer_id	FK → Customer	
status	enum	draft / pending-review / finalized / posted / paid / partial / void
subtotal	decimal	Sum of line totals
total_deposits	decimal	Sum of all deposits
total_tax	decimal	
total_discounts	decimal	
grand_total	decimal	
amount_paid	decimal	
balance_due	decimal (computed)	grand_total - amount_paid
due_date	date	Based on customer payment terms
lock_date	date	After this date, requires elevated permission to modify

Field	Type	Notes
finalized_by	FK → User	Accountant who confirmed
finalized_at	timestamp	
posted_at	timestamp	When officially posted to ledger
created_at	timestamp	
pdf_url	string	Stored PDF version

Invoice Line Items: Mirror of order line items with actual delivered quantities (picked_qty becomes the billed qty).

AI Discrepancy Review (end-of-week finalization):

- Before invoices are "posted," accounting reviews the week's invoices
- AI flags potential issues: pricing anomalies, unusual quantities vs. customer history, missing deposits, duplicate charges, discount policy violations, mismatched pick vs. order quantities
- Accountant reviews flagged items, approves or corrects, then finalizes → invoice is posted to ledger
- *(Exact workflow details pending stakeholder input — see Q33)*

3.5 Payment & Ledger

Payment:

Field	Type	Notes
payment_id	UUID	
customer_id	FK → Customer	
amount	decimal	
method	enum	credit-card / ACH / check / cash
reference_number	string	Transaction ID from Authorize.NET, check #, etc.
status	enum	pending / completed / failed / refunded
received_at	timestamp	
processed_by	FK → User	

Payment Application (links payments to invoices):

Field	Type	Notes
application_id	UUID	

Field	Type	Notes
payment_id	FK → Payment	
invoice_id	FK → Invoice	null if unapplied
amount_applied	decimal	
applied_at	timestamp	
applied_by	FK → User	

Journal Entry (double-entry ledger):

Field	Type	Notes
entry_id	UUID	
entry_date	date	
description	string	Auto-generated from business action
source_type	enum	invoice / payment / credit-memo / adjustment / deposit-return
source_id	UUID	FK to the source record
created_at	timestamp	
created_by	FK → User	

Journal Entry Lines (debit/credit pairs):

Field	Type	Notes
line_id	UUID	
entry_id	FK → JournalEntry	
account_id	FK → ChartOfAccounts	
debit	decimal	
credit	decimal	
memo	string	Optional detail

Chart of Accounts:

Field	Type	Notes
account_id	UUID	
account_code	string	e.g. "1100", "4000"

Field	Type	Notes
account_name	string	e.g. "Accounts Receivable", "Sales Revenue"
account_type	enum	asset / liability / equity / revenue / expense / contra-revenue
is_system	boolean	True = system-managed, can't delete
status	enum	active / inactive

3.6 Route & Delivery

Route:

Field	Type	Notes
route_id	UUID	
route_date	date	
driver_id	FK → User	
truck_id	FK → Truck	
status	enum	planned / loading / in-progress / completed
total_stops	int	
completed_stops	int	
created_by	FK → User	Route planner
created_at	timestamp	

Route Stop:

Field	Type	Notes
stop_id	UUID	
route_id	FK → Route	
order_id	FK → Order	
stop_order	int	Sequence position (1, 2, 3...)
loading_order	int	Reverse of stop_order (last stop loaded first)
status	enum	pending / delivered / rejected / skipped
rejection_reason	string	If rejected

Field	Type	Notes
rejection_note	text	Optional detail
delivered_at	timestamp	Auto-set when invoice printed/sent
gps_lat	decimal	Driver location at delivery
gps_lon	decimal	
location_warning_fired	boolean	True if driver was not near address when printing

Truck:

Field	Type	Notes
truck_id	UUID	
truck_name	string	e.g. "Truck 3"
license_plate	string	
capacity_cases	int	Max cases
status	enum	available / in-use / maintenance

3.7 Users, Roles & Permissions

User:

Field	Type	Notes
user_id	UUID	
username	string	Login
password_hash	string	
full_name	string	
email	string	
phone	string	
role_id	FK → Role	Primary role label
group_ids	FK[] → Group	For policy assignment
status	enum	active / inactive / suspended
preferences	JSON	UI preferences, saved views, favorites

Field	Type	Notes
created_at	timestamp	

Role:

Field	Type	Notes
role_id	UUID	
role_name	string	Label only — actual access is via permissions

Permission:

Field	Type	Notes
permission_id	UUID	
permission_key	string	e.g. "order.create", "invoice.adjust", "discount.apply"
description	string	Human-readable

User/Group Permission Assignment:

Field	Type	Notes
assignment_id	UUID	
assignee_type	enum	user / group / role
assignee_id	UUID	
permission_id	FK → Permission	
granted	boolean	True = allow, False = explicitly deny

Policy (discount caps, adjustment limits, etc.):

Field	Type	Notes
policy_id	UUID	
policy_type	enum	discount-cap / adjustment-limit / order-modification / invoice-lock
policy_name	string	
rules	JSON	Configurable rules specific to policy type
applies_to_type	enum	user / group / role / all
applies_to_id	UUID	

3.8 Notifications & Audit

Notification Rule:

Field	Type	Notes
rule_id	UUID	
trigger_event	enum	invoice-sent / payment-due / delivery-confirmed / credit-limit-warning / review-date / custom
channel	enum	email / sms / whatsapp
timing	JSON	e.g. {"before_days": 3}, {"after_days": 7}, {"on_event": true}
template_id	FK → Template	Message template
applies_to	JSON	Customer groups, tiers, or specific customers
enabled	boolean	

Notification Log:

Field	Type	Notes
log_id	UUID	
rule_id	FK → NotificationRule	
customer_id	FK → Customer	
channel	enum	
message_content	text	Actual message sent
sent_at	timestamp	
delivery_status	enum	sent / delivered / failed / read
reply_content	text	If customer replied (WhatsApp)

Audit Log (system-wide):

Field	Type	Notes
audit_id	UUID	
user_id	FK → User	Who performed the action
action	string	e.g. "order.modified", "discount.applied", "invoice.adjusted"

Field	Type	Notes
entity_type	string	e.g. "order", "invoice", "customer"
entity_id	UUID	
old_value	JSON	Before state
new_value	JSON	After state
reason	text	Required for restricted actions
timestamp	timestamp	
ip_address	string	

3.9 Data Model — Entity Relationship Summary

```
Customer (1) ──→ (N) Location
Customer (1) ──→ (N) Order
Customer (1) ──→ (N) Invoice
Customer (1) ──→ (N) Payment
Order    (1) ──→ (N) OrderLineItem
Order    (1) ──→ (1) Invoice
Invoice  (1) ──→ (N) InvoiceLineItem
Invoice  (N) ←──→ (N) Payment  (via PaymentApplication)
Payment  (1) ──→ (N) PaymentApplication
JournalEntry (1) ──→ (N) JournalEntryLine
JournalEntryLine (N) ──→ (1) ChartOfAccounts
Route    (1) ──→ (N) RouteStop
RouteStop (1) ──→ (1) Order
Route    (N) ──→ (1) User (driver)
Route    (N) ──→ (1) Truck
Category (1) ──→ (N) Subcategory
Subcategory (1) ──→ (N) Product
Product  (1) ──→ (N) ProductVariant
ProductVariant (1) ──→ (N) PackagingUnit
User     (N) ──→ (1) Role
User     (N) ←──→ (N) Permission (via assignment)
```

🟠 PHASE 4: ARCHITECTURE & TECH STACK

4.1 Technology Decisions

Layer	Choice	Rationale
Backend	ASP.NET Core (.NET 8+)	Native on Windows Server, strong typing for ERP complexity, built-in auth (Identity + Policies), SignalR for real-time, Entity Framework ORM, easy Azure migration path later, safe for parallel dev across agents
Database	PostgreSQL + PostGIS	Free (no licensing), native geographic queries (PostGIS) for driver tracking & distance-from-stop calculations, excellent time-series handling for audit logs & order history, range types, table partitioning. Runs well on Windows
Frontend (default)	Razor Pages (server-rendered)	HTML rendered on server, lightweight on client, snappy on cheap Android handhelds, low bandwidth, works on any browser. Used for: accounting, admin, reports, customer profiles, catalog management
Frontend (interactive)	HTMX + Alpine.js (targeted)	Tiny JS footprint (~15-30KB). Used only where client interaction is needed: order builder (live cart), route planner (drag/drop), driver map (live tracking), voice input, camera/photo capture. No full SPA framework
Real-time	SignalR (built into .NET)	Live driver tracking, order status updates, notification pushes — all over WebSocket, no extra dependencies
AI / Voice	Cloud API (OpenAI GPT-class)	Voice order matching, pick sheet OCR (photo → data extraction), invoice discrepancy detection. API calls from server — client just sends audio/photo
Payment Gateway	Authorize.NET SDK	Tokenized vault, card processing, ACH support
Notifications	Twilio (SMS) + WhatsApp Business API + SMTP (email)	All channels from one notification engine on the server
Bluetooth Printing	Web Bluetooth API or native print bridge	Browser-based printing to Bluetooth receipt printers. Exact approach depends on printer model (see Q17)
Maps / Navigation	Google Maps JS API (tracking display) + deep-link to Waze/Google Maps app (turn-by-turn nav)	Map tiles for planner/manager view. Driver launches external nav app for directions
Authentication	ASP.NET Identity + JWT tokens	Cookie auth for desktop/server-rendered pages, JWT for any client-side API calls

4.2 Rendering Strategy — Hybrid Model

Default mode (server-dominant):

```
Browser ↔ ASP.NET Server ↔ PostgreSQL
  │              │
  │  HTML pages  │  Razor Pages render full HTML
  │  ←───────────│  HTMX handles partial page updates (no full reload)
  │              │  Alpine.js for small UI interactions (dropdowns, toggles)
  │              │
  │  API calls   │  Only for: live map, voice recording, photo upload
  │  ←──JSON──→  │  SignalR for push updates
```

- Server does all heavy lifting — DB queries, business logic, rendering
- Client receives ready HTML — minimal JS, fast load, low memory
- HTMX swaps page fragments on user actions (e.g. filtering a table → server renders new rows, HTMX swaps them in) — feels like a SPA but server does the work
- Alpine.js handles micro-interactions (expanding a card, toggle switch, dropdown menus)
- **Field workers (salesperson, driver, warehouse)** get this lightweight experience

Interactive sections (client-side, embedded in server pages):

- Order builder: JS module for cart management, search-as-you-type, voice recording
- Route planner: JS module for drag-and-drop stop ordering, map view
- Live tracker: JS + SignalR for real-time driver map
- Camera capture: JS for pick sheet photo → upload to server for AI processing

4.3 Offline Mode (Optional — Priced Separately)

Without offline (default build):

- Server renders everything, client is thin
- Requires network connection at all times
- Simple, fast, cheaper to build

With offline (add-on module):

- Service worker caches key pages (order form, customer list, product catalog)
- IndexedDB stores: cached product catalog, customer data, orders created offline
- Sync engine queues offline actions → pushes to server when connection returns
- Conflict resolution: server timestamp wins, client notified of any overrides
- Scope: offline ordering + offline delivery confirmation. Accounting/admin always requires connection
- **Engineering impact:** ~25-30% additional dev effort — separate price line in proposal

4.4 Project Structure (High-Level)

```
WholesaleERP/
├── src/
│   ├── WholesaleERP.Web/           ← ASP.NET Core web app (Razor Pages + API controllers)
│   │   ├── Pages/                   ← Razor Pages (server-rendered UI)
│   │   │   ├── Auth/                ← Login, logout, password reset
│   │   │   ├── Sales/               ← Salesperson dashboard, customer list, order wizard
│   │   │   ├── Warehouse/           ← Order queue, pick flow, AI pick review
│   │   │   ├── Delivery/            ← Driver route, stop detail, delivery confirmation
│   │   │   ├── Accounting/          ← Dashboard, invoices, payments, statements
│   │   │   ├── RoutePlanning/       ← Route builder, assignment
│   │   │   ├── Admin/               ← Users, roles, policies, catalog, settings
│   │   │   ├── Reports/             ← Report builder, saved reports, dashboards
│   │   │   └── Shared/              ← Layouts, partials, components
│   │   ├── wwwroot/                 ← Static assets
│   │   │   ├── js/                  ← HTMX, Alpine.js, custom modules (order builder, map, voice)
│   │   │   ├── css/                 ← Stylesheets
│   │   │   └── images/
│   │   ├── Hubs/                    ← SignalR hubs (live tracking, notifications)
│   │   └── Program.cs              ← App startup, DI config
│   │
│   ├── WholesaleERP.Core/          ← Domain models, interfaces, business rules
│   │   ├── Entities/                ← Customer, Product, Order, Invoice, etc.
│   │   ├── Interfaces/              ← Repository + service interfaces
│   │   ├── Enums/
│   │   └── ValueObjects/            ← Address, Money, DateRange, etc.
│   │
│   ├── WholesaleERP.Application/   ← Business logic / use cases
│   │   ├── Orders/                  ← Order creation, modification, cancellation
│   │   ├── Invoicing/               ← Invoice generation, finalization, AI review
│   │   ├── Payments/                ← Payment processing, application, reconciliation
│   │   ├── Accounting/              ← Ledger entries, chart of accounts, reports
│   │   ├── Routing/                 ← Route building, assignment, tracking
│   │   ├── Warehouse/               ← Pick assignment, AI pick sheet, approval
│   │   ├── Notifications/           ← Notification engine (triggers, channels, templates)
│   │   ├── Catalog/                 ← Product management, visibility, pricing
│   │   ├── Auth/                    ← User management, permissions, policies
│   │   └── AI/                      ← Voice matching, OCR, discrepancy detection
│   │
│   ├── WholesaleERP.Infrastructure/ ← External integrations & data access
│   │   ├── Data/                    ← EF Core DbContext, migrations, repositories
│   │   ├── Payment/                 ← Authorize.NET integration
```

```
│   │   ├── Messaging/              ← Twilio SMS, WhatsApp, email (SMTP)
│   │   ├── AI/                     ← OpenAI API client (voice, OCR, discrepancy)
│   │   ├── Maps/                   ← Google Maps API integration
│   │   └── Printing/               ← Bluetooth print bridge
│   │
│   └── WholesaleERP.Tests/         ← Unit + integration tests
│       ├── Core.Tests/
│       ├── Application.Tests/
│       └── Integration.Tests/
│
├── docs/                           ← Design docs, API docs, deployment guide
├── scripts/                        ← DB migrations, seed data, deployment scripts
└── README.md
```

> **Clean Architecture layers:** Web → Application → Core ← Infrastructure. Core has zero dependencies. Application contains all business logic. Infrastructure handles DB, APIs, external services. Web is the thin UI layer.

4.5 Parallel Development Splits

> When developing, the project splits naturally across multiple coding agents:

Agent	Module	Dependencies
Agent 1	Core entities + DB schema + migrations	None — starts first
Agent 2	Auth, Users, Roles, Permissions	Core entities
Agent 3	Product Catalog + Pricing	Core entities
Agent 4	Order flow + Order UI (sales pages)	Core + Catalog
Agent 5	Warehouse flow + Pick UI + AI OCR	Core + Orders
Agent 6	Routing + Driver UI + Maps	Core + Orders
Agent 7	Invoicing + Payments + Ledger + Accounting UI	Core + Orders
Agent 8	Notifications engine + all channels	Core
Agent 9	Admin panel + Policies + Reports	Core + all above

> Agents 1 must finish first (shared foundation). Then 2-4 can run in parallel. Then 5-8 in parallel. Agent 9 last (ties everything together).

4.6 Server Requirements (Preliminary)

- **OS:** Windows Server 2019+ (or Windows 10/11 for dev)
- **.NET Runtime:** .NET 8+ (self-contained deployment possible — no runtime install needed)
- **PostgreSQL:** 15+ with PostGIS extension
- **RAM:** 8GB minimum, 16GB recommended
- **Storage:** 50GB+ (depends on data volume, images, PDFs)
- **Network:** LAN access for all users. Internet required for: AI API calls, payment processing, notifications (SMS/email), map tiles
- *(Exact specs TBD pending client server info)*

🗒️ Stakeholder questions pending: [Q17] printer models, [Q32] order/invoice separation, [Q33] invoice finalization workflow, server specs

---

🔐 PHASE 5: SECURITY & ACCESS CONTROL

5.1 Network Architecture & Isolation

Core principle: The app server and database sit on a **private network**. No direct public internet access to the app or DB.

```
┌──────────────────────────────────────────────────────────┐
│  PRIVATE NETWORK (Office LAN / VPN)                      │
│                                                          │
│   ┌──────────────┐     ┌──────────────┐                 │
│   │  App Server  │────→│  PostgreSQL  │                 │
│   │  ASP.NET Core │     │  (DB)        │                 │
│   └──────┬───────┘     └──────────────┘                 │
│          │                                               │
│   Desktop users connect directly on LAN                  │
│                                                          │
└──────────┼──────────────────────────────────────────────┘
           │
           ┌──────┴──────┐
    ┌──────────────┐
    │   Firewall   │  ← Standard firewall + antivirus on server
    │  + Reverse   │  ← Only specific ports open
    │   Proxy      │  ← Rate limiting, DDoS protection
    └──────┬──────┘
           │
           ┌──────┴──────┐
    ┌──────────────┐
    │  Mobile      │  ← Registered devices only
    │  Devices     │  ← VPN or certificate-pinned connection
    │  (Field)     │  ← App-layer auth on top
    └──────────────┘
```

Key rules:

- Database is **never** exposed to any external network — app server only
- App server listens on private LAN — desktop users connect directly
- Mobile/field devices connect through firewall — only the app's specific port is open
- **Logically isolated:** Only the app can talk to the server — no general browser access from public internet
- Standard **firewall** on server (Windows Firewall + network firewall)
- **Antivirus** on server (Windows Defender or enterprise AV)

5.2 Mobile Device Security

> Mobile devices (Android handhelds) are **company-owned and handed over to workers**. This is a controlled device scenario.

Device Verification:

- Each mobile device is **registered** in the admin panel — device ID, assigned user, approved date

- First connection: admin generates a device registration code → worker enters it on the device → device is paired to that user account
- Only registered devices can authenticate — unregistered devices are rejected at login
- Admin can revoke a device instantly (lost/stolen/employee leaves) → device is blocked immediately

Easy to use, hard to breach:

- After device registration, worker logs in with **simple PIN or biometric** (fingerprint) — not full username/password every time
- Full credentials required only on first setup or after device revocation
- Session stays alive during work hours — no annoying re-logins (configurable session duration per role)
- Auto-lock after configurable idle period — requires PIN/biometric to resume (not full re-login)

Data protection on device:

- No sensitive data cached in plain text on device — any local storage is encrypted
- If offline mode is enabled: local DB is encrypted at rest
- Remote wipe capability — admin can clear app data on a lost device

5.3 Desktop Access — IP-Based Permissions

- Desktop users (accounting, admin, managers) connect from **office network only** by default
- **IP allowlist** configured in admin panel — only approved IP addresses / ranges can access the app
- Attempts from unapproved IPs are blocked and logged

Optional bypass (stakeholder decision — adds cost):

- SSO integration (Azure AD, Google Workspace, or company LDAP/Active Directory) — if enabled, users can authenticate from outside the office via SSO + MFA
- VPN requirement for remote access — connect to office VPN first, then access app normally
- *(Ask stakeholder: do they need remote access outside the office, or is IP-lock sufficient? — see Q35)*

5.4 Authentication Layers

Layer	Method	Applies to
Device registration	Device ID + registration code	Mobile only
Primary login	Username + password	All users (first time / new session)
Quick unlock	PIN or biometric	Mobile users (resume after idle)
IP allowlist	Approved IPs only	Desktop users (default)
SSO + MFA (optional)	Azure AD / LDAP + authenticator app	All users (optional bypass for IP restriction)
Session management	Configurable timeout per role	All users

5.5 Visibility & Data Access Policy

> **All data visibility is policy-managed** — configured by admin in the control panel.

- **Who sees what** is NOT hardcoded by role — admin sets policies per user, per group, or per role
- Default sensible policies out of the box:
  - Salesperson sees only their assigned customers, orders, and accounts
  - Warehouse worker sees only orders assigned to them for picking
  - Driver sees only their assigned route and stops
  - Accounting sees all customers, all financial data
  - Manager sees everything — all users, all data, all reports
  - Admin has full system access including security settings
- Admin can override any default — e.g. grant a senior salesperson visibility into another salesperson's accounts, or restrict an accountant to specific customer groups
- **Policy engine** — same system that controls discount caps and adjustment limits also controls data visibility

5.6 Additional Security Measures

- **HTTPS everywhere** — even on local network, all traffic encrypted (self-signed cert or internal CA)
- **Rate limiting on login** — 5 failed attempts → account locked for 15 min (configurable)
- **Tamper-proof audit log** — audit entries are append-only; even system admins cannot delete or modify them. Stored in a separate DB schema with restricted write access
- **Reason codes required** for sensitive actions — adjusting an invoice, overriding a pick, changing a credit limit, modifying a locked record → must select reason from predefined list + optional note
- **Payment data (PCI)** — raw card numbers never touch the app; all card data goes directly to Authorize.NET via their SDK. App only stores tokenized references ("Visa ending 4242")
- **Password policy** — minimum length, complexity rules, expiration period (all configurable in admin)
- **Active session dashboard** — admin can see all logged-in users, their device, IP, and last activity. Can force-logout any user remotely

5.7 Optional: Full Private Network Lockdown (Zero Public Exposure)

> **Optional module — adds cost.** For clients who want maximum network isolation.

Everything runs on private network — nothing public-facing:

- App server: private LAN only, no public IP, no public DNS record
- Database: private LAN only (already default)
- **No public DNS entries** — the app has no resolvable domain name on the public internet. Internal DNS only (e.g. `erp.company.local` resolved by internal DNS server)

- All external services (AI API, Authorize.NET, Twilio, Google Maps) accessed through the server outbound only — no inbound ports open to the internet

VPN-only access for all users:

- **Desktop users:** Must connect to company VPN to access the app — even from inside the office (or LAN-only if on-site)
- **Mobile/field users:** VPN client installed on Android handhelds — auto-connects when app opens. All traffic tunnels through company VPN before reaching the app server
- VPN server runs on-prem (WireGuard, OpenVPN, or Windows SSTP) or through a managed VPN gateway
- VPN certificates per device — tied to device registration. Revoke device = revoke VPN cert = instant lockout

DNS isolation:

- Internal DNS server resolves `erp.company.local` (or chosen hostname) to the app server's private IP
- No A/CNAME records exist in any public DNS — the app is invisible to the outside world
- Mobile devices use VPN's DNS resolver — resolves internal hostnames only when connected

Network diagram with full lockdown:

```
┌──────────────────────────────────────────────────────────┐
│  PRIVATE NETWORK (Zero public exposure)                  │
│                                                          │
│   ┌──────────────┐     ┌──────────────┐  ┌────────────┐ │
│   │  App Server  │────→│  PostgreSQL  │  │ Internal   │ │
│   │  (no public  │     │  (no public  │  │ DNS Server │ │
│   │   IP/DNS)    │     │   exposure)  │  │ *.local    │ │
│   └──────┬───────┘     └──────────────┘  └────────────┘ │
│          │                                               │
│   Desktop users (on LAN) ← direct access                 │
│                                                          │
│   ┌──────────────┐                                       │
│   │  VPN Server  │ ← Only inbound entry point            │
│   │  (WireGuard/ │                                       │
│   │   OpenVPN)   │                                       │
│   └──────┬───────┘                                       │
│          │                                               │
└──────────┼───────────────────────────────────────────────┘
           │ (encrypted tunnel)
    ┌──────┴───────┐
    │  Mobile      │  VPN auto-connect
    │  Devices     │  Device cert + PIN
    │  (Field)     │  Internal DNS via VPN
    └──────────────┘
```

Trade-offs:

- **Pro:** Maximum security — app is invisible to the internet, zero attack surface
- **Con:** VPN adds setup complexity, slight latency for mobile users, VPN server is an extra component to maintain
- **Con:** External services (AI, payments, notifications) still need outbound internet — server must have outbound access while blocking all inbound

> This is a **priced optional module** — the default build uses IP allowlist + device registration (already very secure). Full VPN lockdown is for clients who want enterprise-grade network isolation.

🗒️ Stakeholder questions:

- **[Q35]** Do users need remote access from outside the office network? If yes — SSO + MFA, VPN, or both? (Affects pricing)

- **[Q36]** Are the Android handhelds company-managed (MDM like Intune) or just handed out as-is? MDM would add device-level security

---

Document last updated: Session 2 — February 25, 2026 | v1.0 — Architecture complete